AES SOLSVED EXAMPLE

a. 16 bit key for round zero

- **Plaintext:** `32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34`
- **Round 0 Key:** `2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C`

1. Now convert this key into 4*4 matrix and XOR Round key and plaintext

w0	w1	w2	w3
2B	28	AB	09
7E	AE	F7	CF
15	D2	15	4F
16	A6	88	3C

P 0	P 1	P 2	P 3
32	88	31	E0
43	5A	31	37
F6	30	98	07
A8	8D	A2	34

2. Now apply XOR on round 0 key and plain text for making key for round 1

Plaintext Byte	**⊕**	**Key Byte**	**Result (Hex)**
32(001100102)	⊕	2B (001010112)	19 (000110012)
43 (010000112)	⊕	7E (011111102)	3D (001111012)
F6 (111101102)	⊕	15(000101012)	E3 (111000112)
A8 (101010002)	⊕	16 (000101102)	BE (101111102)
After XOR This is the result this will be round 1 key			
W3	**W4**	**W5**	**W6**
19	A0	9A	E9
3D	F4	6F	F8
E3	E2	8D	48
BE	2B	20	08

3. **Now apply s-box function after round 1 key this will be called step no 1**

The Process: "Search and Replace"

You take your matrix and look at each byte (two characters, like `19` or `3D`) one by one.

- **(Split):** Take the first digit as the **Row** and the second digit as the **Column**.
- **(Find):** Go to that spot in the S-Box table.
- **(Swap):** Throw away your old number and write down the new number found in the table.

The Conversion Process

We look up each byte. For example, for **A0**, we go to Row **A**, Column **0** in the S-Box table, which gives us **E0**.

- **Column 0:** `19` → **D4**, `3D` → **27**, `E3` → **11**, `BE` → **AE**
- **Column 1:** `A0` → **E0**, `F4` → **BF**, `E2` → **98**, `2B` → **F1**
- **Column 2:** `9A` → **B8**, `6F` → **A8**, `8D` → **5D**, `20` → **B7**
- **Column 3:** `E9` → **1E**, `F8` → **41**, `48` → **52**, `08` → **30**

Here is the complete result after applying s box substitution

Column 0	**Column 1**	**Column 2**	**Column 3**
D4	**E0**	**B8**	**1E**
27	**BF**	**A8**	**41**
11	**98**	**5D**	**52**
AE	**F1**	**B7**	**30**

Now that we have the **SubBytes** result, we move to the next step: **ShiftRows**.

Think of this step as "shuffling the rows." It is very simple to follow because we don't change any numbers; we just slide them to the left.

1. The Rules of Shift-Rows

Each row in our 4 * 4 matrix has a different "shift" rule:

- **Row 0:** No shift (stays the same).
- **Row 1:** Shift **1** position to the left.
- **Row 2:** Shift **2** positions to the left.
- **Row 3:** Shift **3** positions to the left.

Note: When a byte "falls off" the left side, it simply wraps around to the right side of the same row.

Row	Current Row (After S-Box)	Action	New Row (After Shift)
Row 0	`D4 E0 B8 1E`	No Change	**`D4 E0 B8 1E`**
Row 1	`27 BF A8 41`	Shift 1 Left	**`BF A8 41 27`**
Row 2	`11 98 5D 52`	Shift 2 Left	**`5D 52 11 98`**
Row 3	`AE F1 B7 30`	Shift 3 Left	**`30 AE F1 B7`**

The Complete Matrix (After ShiftRows)

Now, our matrix looks like this:

Column 0	Column 1	Column 2	Column 3
D4	**E0**	**B8**	**1E**
BF	**A8**	**41**	**27**
5D	**52**	**11**	**98**
30	**AE**	**F1**	**B7**

let's step through the **Key Expansion** process. We will take our original **Round 0 Key** and transform it into the **Round 1 Key** using the g-function.

1. The Starting Point (Round 0 Key)

Our original key is divided into four 32-bit words (w0, w1, w2, w3):

- w0 = [2B, 7E, 15, 16]
- w1 = [28, AE, D2, A6]
- w2 = [AB, F7, 15, 88]
- **w3 = [09, CF, 4F, 3C]** (We will use this for the g-function)

2. Applying the g-function to w3

The g-function has three mini-steps to scramble the word:

Step A: RotWord (Rotate)

Take the first byte and move it to the end.

- Input: `[09, CF, 4F, 3C]`
- Result: `[CF, 4F, 3C, 09]`

Step B: SubWord (S-Box)

Replace each of these bytes using the S-Box table we used earlier.

- `CF` to **`8A`**
- `4F` to **`84`**
- `3C` to **`EB`**
- `09` to **`01`**
- Result: `[8A, 84, EB, 01]`

Step C: Rcon (Round Constant) (01 00 00 00)

XOR the **first byte only** with the Round Constant for Round 1, which is `01`.

- `8A` + `01` = **`8B`**
- The rest stay the same: `84, EB, 01`
- **Final g(w3) Result: `[8B, 84, EB, 01]`**

3. Generating the Round 1 Words (w_4 to w_7)

Now we follow the XOR pattern to build the new key:

- **To get w_4:** $w_0 \oplus g(w_3)$
  - `[2B, 7E, 15, 16]` $\oplus$ `[8B, 84, EB, 01]` = `[A0, FA, FE, 17]`
- **To get w_5:** $w_1 \oplus w_4$
  - `[28, AE, D2, A6]` $\oplus$ `[A0, FA, FE, 17]` = `[88, 54, 2C, B1]`
- **To get w_6:** $w_2 \oplus w_5$
  - `[AB, F7, 15, 88]` $\oplus$ `[88, 54, 2C, B1]` = `[23, A3, 39, 39]`
- **To get w_7:** $w_3 \oplus w_6$
  - `[09, CF, 4F, 3C]` $\oplus$ `[23, A3, 39, 39]` = `[2A, 6C, 76, 05]`

4. The Final Round 1 Key

Combining these four words, our complete **Round 1 Key** is:

`A0 FA FE 17 88 54 2C B1 23 A3 39 39 2A 6C 76 05`

Summary Table

Word	Value	Calculation Method
w_4	`A0 FA FE 17`	$w_0 \oplus g(w_3)$
w_5	`88 54 2C B1`	$w_1 \oplus w_4$
w_6	`23 A3 39 39`	$w_2 \oplus w_5$
w_7	`2A 6C 76 05`	$w_3 \oplus w_6$

THE FINAL ROUND TO CREATE NEW STATE BYTE KEY FOR NEXT ROUND

AddRoundKey step for Round 1 properly. We will XOR every single byte of your **ShiftRows Matrix** with the **Round 1 Key** you generated.

Key after shift row

Column 0	Column 1	Column 2	Column 3
D4	E0	B8	1E
BF	A8	41	27
5D	52	11	98
30	AE	F1	B7

Key after applying g function

Word 4 (w4)	Word 5 (w5)	Word 6 (w6)	Word 7 (w7)
A0	88	23	2A
FA	54	A3	6C
FE	2C	39	76
17	B1	39	05

The XOR Calculation Process the end of round 1

To get the result, we calculate: **shift row column XOR Key Byte = New State Byte**.

Column 0	Column 1	Column 2	Column 3
74	68	9B	34
45	FC	E2	4B
A3	7E	28	EE
27	1F	C8	B2

This final result is now ready to be used as the input for **Round 2**